Your first five minutes

Module 5: Skill Matching

Explain one job match and try temporary improvements with Career GPS.

You start with an empty folder. Your Python backend handles logic; your React frontend is the browser screen. Use ordinary ChatGPT Free or Claude Free for help. You need no completed project, autonomous agent or teammate server. JSON means text with named data fields; it is how your parts exchange results.

A concrete example

Asha has a reviewed Python claim. Job A requires Python and prefers SQL. With weights 2 and 1, she covers 2/3 of the weighted requirements: 66.7%. Adding hypothetical SQL projects 100%, but must not add SQL to her real saved profile.

Where your work fits

Module 1 supplies a reviewed JD; Module 3 supplies a saved profile. A resume can enter through Module 3's reviewed import. Company readiness is a separate optional comparison supplied by Module 4's shared arithmetic.

The full journey is: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. The last two are different comparisons: company-wide readiness and one specific job.

1. Load pair	**2. Review names**	**3. Score groups**
4. Explain gaps	**5. Try a copy**	**6. Reset**

Do this first

- 1. Install the tools in the next section and create your own project folder.
- 2. Make the tiny health endpoint work. An API is how the browser asks your Python program for data; an endpoint is one address on that API.
- 3. Build one milestone at a time. You type/copy code, save files, run commands and inspect errors yourself. This PDF contains every required fictional input and agreed contract.

Read the requirements next, then start installing. Keep this PDF open beside VS Code. The PDF-only library is a reading resource; do not clone a finished app or upload your source into that library.

What counts as complete?

Organizer requirements

- [ ] Compare one candidate profile or resume-derived skill list against one specific extracted JD, not a company in general.
- [ ] Return a score plus matched and missing skills. Prioritize an understandable gap list, not just the number.
- [ ] Handle naming variation with exact/alias/fuzzy review logic. The organizer allows simple matching; an LLM is not mandatory for this part.
- [ ] Run independently with provided fictional JSON and show different results for different candidates/jobs. Later verify the organizer pack and the integrated journey.

ProofPath project requirements

- [ ] Explain JD quote -> candidate claim or absence -> weighted contribution -> gap -> useful action.
- [ ] Preserve mandatory/preferred weights, AND/OR alternatives, and uncertain evidence that needs review.
- [ ] Implement Career GPS using copies and the SAME scoring function; support reset, frozen input versions and useful single-change recommendations.
- [ ] Keep named-JD matching separate from company readiness and never turn evidence confidence into a proficiency number.

Optional later work

Embeddings, complex recommendation models and cloud deployment are optional. A spelling suggestion must not silently become a confirmed match. Reusable transparent logic comes before sophisticated similarity.

Be honest about missing resources

The original six JDs, four resumes and real company benchmarks have not been supplied. All examples here are invented. Build and test with them now; mark official-pack and authorized cloud acceptance pending. Synthetic success is not official-data acceptance.

An LLM is a language model that interprets text; Supabase is a cloud database service. The brief's JSON was a starting point. These manuals agree a simpler TEAM-V1 contract, different from the completed app's 1.0 format. The Scrum Master must build the explicit adapter described near the end; do not claim they already plug together.

Install your tools and open a folder

- 1. Install Python 3.12 from python.org/downloads/windows. Enable its PATH option. Python runs the backend: the program that reads files, validates data and calculates results.
- 2. Install Node.js 24 LTS from nodejs.org/en/download. It includes npm, a tool that installs JavaScript libraries. React runs the frontend: the screen in the browser.
- 3. Install VS Code from code.visualstudio.com/download and Git for Windows from git-scm.com/downloads/win. Git records versions of your work; GitHub stores a copy online.
- 4. Reopen PowerShell. Run the version checks below. If a command is not found, restart the terminal and check installation/PATH before continuing.

```
python --version
node --version
npm.cmd --version
git --version
```

- 5. Choose a location such as Documents. In PowerShell create your own project; code . opens it in VS Code. If code is unavailable, use VS Code -> File -> Open Folder.

```
mkdir radix-skill-matching
cd radix-skill-matching
code .
mkdir backend
mkdir samples
mkdir backend/tests
python -m venv .venv
```

In VS Code use Terminal -> New Terminal and choose PowerShell. Every command below says whether it starts at the project root or frontend/. The root is the folder containing backend/ and frontend/. .venv is an isolated Python dependency folder, not a file you write. Commands use its full Python path so you do not need to change PowerShell execution policy.

In the Explorer panel, right-click a folder -> New File. Create backend/__init__.py as an empty file. Names matter: app.py is a Python file; App.jsx is a React file. Save with Ctrl+S.

Make your first working backend

From the project root install the libraries. FastAPI creates API routes; Uvicorn runs the server; Pydantic checks data types; pytest runs tests; httpx sends HTTP requests and supports local API tests. python-multipart handles uploaded forms.

A backtick (`) at the end of a command continues it on the next line in PowerShell. Copy the whole command, including its continuation lines; leave no spaces after the backtick.

```
.\.venv\Scripts\python.exe -m pip install fastapi "uvicorn[standard]" pydantic `
  python-multipart httpx pytest python-dotenv pypdf python-docx
```

pypdf reads PDF text; python-docx reads Word files. Modules 4/5 mostly use JSON, but the shared install list is harmless. This is installation, not your finished module. Create backend/app.py and type this small health check:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:5173"],
    allow_methods=["GET", "POST", "PATCH", "DELETE"],
    allow_headers=["Content-Type"],
)

@app.get("/api/team/v1/health")
def health():
    return {"status": "ok"}

.\.venv\Scripts\python.exe -m uvicorn backend.app:app --reload --host 127.0.0.1 `
  --port 8000
```

Keep this terminal running. Open http://127.0.0.1:8000/api/team/v1/health and expect {"status":"ok"}. Also open http://127.0.0.1:8000/docs. Swagger /docs is an automatically generated form for calling your endpoints; choose Try it out, enter fields, then Execute.

CORS is the browser's cross-origin permission check. Your screen on port 5173 needs permission to call port 8000. Use the exact 127.0.0.1 addresses throughout; localhost is a different origin. This is local single-user development, not a production authentication design.

Start React and use web AI properly

Open a SECOND terminal at the project root; leave Python running in the first. Vite creates a small React project. Choose React/JavaScript if prompted; decline optional extra features for now.

```
npm.cmd create vite@latest frontend -- --template react
cd frontend
npm.cmd install
npm.cmd run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

Open http://127.0.0.1:5173/. Seeing the Vite/React starter is your first frontend success. Later replace frontend/src/App.jsx and App.css. Never run the create command again over your work. If scripts are blocked, the commands above use npm.cmd directly without bypassing device policies.

Your browser tutor cannot see your computer

Open a normal ChatGPT or Claude web chat. Upload this PDF if your account permits it. Otherwise paste the purpose, current milestone and relevant contract pages. Free accounts may limit uploads or messages; work in small steps and keep local notes so a new chat can continue.

Ask for one or two named files at a time. Copy the returned code into the exact files yourself, save, run the command yourself, and paste the safe output back. Never say 'read my repo' unless you actually paste/upload its files. No tool subscription or IDE agent is assumed.

Copy this first prompt

I am building RADIX module 5, Skill Matching, from scratch on Windows with basic programming knowledge. You are my web-chat tutor, not an agent on my computer. I will create files, copy code and run commands myself. Use this PDF's TEAM-V1 contract exactly. Work one milestone at a time, name each file, explain the code and wait for my actual result. Ask me to paste missing files or safe errors. Do not request keys, invent official data or assume an existing implementation.

At each milestone, also paste its Build requirements and the referenced full JSON. If AI proposes a different field or endpoint, compare it with this PDF before copying. Keep working versions in Git as you progress.

Understand the common data language

JSON is text made of named fields: strings use double quotes, lists use [], objects use {}, true/false are booleans and null means unknown/not present. Save JSON files as UTF-8. JSON does not allow comments or trailing commas. A schema is the agreed set of fields, types and rules.

When copying a PDF example, a viewer may insert line breaks inside a long quoted string. Join those visual wraps into one line, keeping literal \n escapes. Check your saved file with: python -m json.tool samples/profile.json (use your actual filename). A parse error must be fixed before testing the API.

Every object below is a complete fictional fixture, not a recording of live Groq output. IDs are short teaching examples; your backend should generate unique IDs. Compare meaning, references and arithmetic, not literal generated IDs or exact model confidence.

One category per skill

COD = Coding; DSA = Data Structures & Algorithms; OOD = Object-Oriented Design

APTI = Aptitude; COMM = Communication; AI = AI

CLOUD = Cloud; SQL = SQL; SWE = Software Engineering

SYSD = System Design; NETW = Networking; OS = Operating Systems

OTHER is allowed for a skill outside the twelve. It is not a thirteenth rating. Named technologies stay named: the category COD alone does not mean Python, and Java is not JavaScript.

Skill fields: all nine are present

- skill_id, skill_name and category_code: nonblank strings; IDs unique within a record. Category must be one of the codes above or OTHER.
- evidence: a short exact source quote for a document claim, or a clearly labelled manual reason. confidence: high, medium or low; this is NOT a 1-10 competency rating.
- source_type: document, manual or hypothetical. source_id: a source record ID or null for manual/hypothetical. source_page: positive integer if genuinely known, otherwise null (often Word).
- claim_status: claimed or needs_review. Keep negative/conflicting evidence for review. All claims remain unverified; never imply that review proves ability.

For document claims, the source ID must exist and the quote must match its source text (normalize whitespace only). Hypothetical claims are allowed only inside a temporary scenario, never in saved real profiles. Keep original source files private unless fictional and safe to share.

Extraction and profile rules

Extraction: what Modules 1 and 2 hand off

schema_version is TEAM-V1. id is a unique string; revision is an integer >=1. source_type is jd or resume; source_file is a safe basename; source_text is the actual full extracted text. company/role may be null; name/email may be empty. skills is a list of the common Skill objects.

education, projects, experience, internships, hackathons, certifications and achievements are lists of supported strings, empty when absent. JD records normally leave them empty. Resume requirement_groups is empty. review_status is needs_review or reviewed; extraction_method is groq, manual or rules. warnings is a list of strings. A Groq result starts needs_review, never self-approved.

Each JD requirement group has requirement_id (unique string), skill_ids (nonempty list referencing the same record), operator (AND/OR), requirement_type (mandatory/preferred/unspecified) and evidence (an exact quote). Keep duplicate group members out. Ambiguous wording needs a warning and review, not fabricated certainty.

Profile: what Module 3 saves

The full profile example below defines every field. name is nonblank; email may be empty but must have an email shape if present. Background fields are lists. skills use the same nine fields. ratings is an object with exactly the twelve codes, each null or a finite number 1-10; booleans, NaN and infinity are invalid.

documents is a list of source_id/source_type/source_file/source_text objects. Imported resume id becomes its document source_id. Every document skill points into this list. cv_file is that private source_id, not a public URL or a path on someone else's PC; it may be null. Actual original bytes need an attachment before download can work.

Only the person explicitly enters self-ratings. Unknown stays null. Importing a quote, increasing confidence or completing a hypothetical scenario must never alter a saved real rating. Extra unknown JSON fields should be rejected, so contract mismatches are noticed early.

Your full Profile input - no other server needed

Create samples/profile.json from this complete fictional record. It contains only a Python claim and all ratings Unknown. Matching named requirements must work without twelve numeric ratings or company benchmarks.


```
{
  "schema_version": "TEAM-V1",
  "id": "candidate-1",
  "revision": 1,
  "name": "Asha Example (fictional)",
  "email": "asha@example.test",
  "education": ["Fictional College - BSc Computing"],
  "skills": [
    {"skill_id": "cv-python", "skill_name": "Python", "category_code": "COD",
  "evidence": "Skills: Python", "confidence": "medium", "source_type": "document",
  "source_id": "resume-1", "source_page": null, "claim_status": "claimed"}
  ],
  "hackathons": [],
  "internships": [],
  "certifications": [],
  "preferred_roles": ["Junior Developer"],
  "projects": ["Built a Python quiz."],
  "experience": [],
  "achievements": [],
  "cv_file": "resume-1",
  "ratings": {"COD": null, "DSA": null, "OOD": null, "APTI": null, "COMM": null, "AI":
   null, "CLOUD": null, "SQL": null, "SWE": null, "SYSD": null, "NETW": null, "OS":
  null},
  "documents": [
    {"source_id": "resume-1", "source_type": "resume", "source_file": "cv_a.docx",
  "source_text": "Name: Asha Example (fictional)\nEmail: asha@example.test\nSkills:
  Python\nProjects: Built a Python quiz.\nEducation: Fictional College - BSc
  Computing"}
  ]
}
```

Your full reviewed JD input

Create samples/jd_reviewed.json using this whole object. It is a hand-labelled reviewed fixture: Python is mandatory, SQL preferred. Its source quotes and skill IDs are internally linked; do not rename them in only one place.


```
{
  "schema_version": "TEAM-V1",
  "id": "jd-1",
  "revision": 1,
  "source_type": "jd",
  "source_file": "jd_a.docx",
  "source_text": "Company: Synthetic Practice Co\nRole: Junior Developer\nRequired:
  Python\nPreferred: SQL",
  "company": "Synthetic Practice Co",
  "role": "Junior Developer",
  "name": "",
  "email": "",
  "skills": [
    {"skill_id": "jd-python", "skill_name": "Python", "category_code": "COD",
  "evidence": "Required: Python", "confidence": "medium", "source_type": "document",
  "source_id": "jd-1", "source_page": null, "claim_status": "claimed"},
    {"skill_id": "jd-sql", "skill_name": "SQL", "category_code": "SQL", "evidence":
  "Preferred: SQL", "confidence": "medium", "source_type": "document", "source_id":
  "jd-1", "source_page": null, "claim_status": "claimed"}
  ],
  "requirement_groups": [
    {"requirement_id": "req-1", "skill_ids": ["jd-python"], "operator": "AND",
  "requirement_type": "mandatory", "evidence": "Required: Python"},
    {"requirement_id": "req-2", "skill_ids": ["jd-sql"], "operator": "AND",
  "requirement_type": "preferred", "evidence": "Preferred: SQL"}
  ],
  "education": [],
  "projects": [],
  "experience": [],
  "internships": [],
  "hackathons": [],
  "certifications": [],
  "achievements": [],
  "review_status": "reviewed",
  "extraction_method": "manual",
  "warnings": ["Fictional expected fixture; not a live Groq result."]
}
```

Complete matching response

Create samples/expected_match.json if you want a comparison file. This is the full expected 66.7% result. Score is requirement coverage, not a probability of hiring or a measured proficiency.


```
{
  "schema_version": "TEAM-V1",
  "jd_source_file": "jd_a.docx",
  "jd_id": "jd-1",
  "jd_revision": 1,
  "profile_id": "candidate-1",
  "profile_revision": 1,
  "match_score": 66.7,
  "status": "complete",
  "matched_skills": ["Python"],
  "missing_skills": ["SQL"],
  "decisions": [
    {"skill_name": "Python", "matched": true, "jd_evidence": "Required: Python",
  "candidate_evidence": "Skills: Python", "points": 66.6667, "reason": "Exact reviewed
   skill name."},
    {"skill_name": "SQL", "matched": false, "jd_evidence": "Preferred: SQL",
  "candidate_evidence": null, "points": 0.0, "reason": "No reviewed claim found."}
  ],
  "group_contributions": [
    {"requirement_id": "req-1", "operator": "AND", "requirement_type": "mandatory",
  "weight": 2, "coverage": 1.0, "points": 66.6667},
    {"requirement_id": "req-2", "operator": "AND", "requirement_type": "preferred",
  "weight": 1, "coverage": 0.0, "points": 0.0}
  ],
  "notice": "Coverage of reviewed extracted requirements; not verified ability or
  hiring probability."
}
```

Career GPS response and optional readiness

For the same uploaded files, send profile_revision=1, jd_revision=1, skill_overrides_json=["SQL"] and category_overrides_json={}. Omit company_file. The complete expected ScenarioResult is:

```
{
  "schema_version": "TEAM-V1",
  "hypothetical": true,
  "profile_revision": 1,
  "jd_revision": 1,
  "baseline_match_score": 66.7,
  "projected_match_score": 100.0,
  "match_delta": 33.3,
  "baseline_readiness_score": null,
  "projected_readiness_score": null,
  "readiness_delta": null,
  "recommendations": [
    {"action": "Review SQL evidence; demonstrate or develop it if needed.", "skill":
  "SQL", "projected_match_score": 100.0, "gain": 33.3}
  ],
  "notice": "Temporary scenario only. The saved profile is unchanged; no company was
  supplied."
}
```

baseline_readiness_score/projected_readiness_score are nullable numbers in TEAM-V1, not nested result objects. If a company is supplied, use the full twelve-category formula below and return numbers only when all categories are known.

For company testing create a Company with schema_version TEAM-V1, id synthetic-company-1, company Synthetic Practice Co, source_mode synthetic, a clear invented-data notice, and levels containing all twelve codes at 8. Use the complete JSON on the next page; do not guess real standards.

Optional GPS Company fixture


```
{
  "schema_version": "TEAM-V1",
  "id": "synthetic-company-1",
  "company": "Synthetic Practice Co",
  "source_mode": "synthetic",
  "notice": "Invented teaching values. Not official company expectations.",
  "levels": {"COD": 8, "DSA": 8, "OOD": 8, "APTI": 8, "COMM": 8, "AI": 8, "CLOUD": 8,
  "SQL": 8, "SWE": 8, "SYSD": 8, "NETW": 8, "OS": 8}
}
```


For this optional test, copy profile.json to rated_profile.json and deliberately replace all twelve null ratings with 4. Baseline readiness is 50.0. Hypothetical COD=8 projects 54.2; adding named SQL changes JD matching but leaves readiness 50.0. Changing only a category level must not add a named skill.

Use the same pure readiness function as Module 4: for every category, attainment=min(candidate/required,1), shortfall=max(required-candidate,0), gap=candidate<required. The row contributes 100*attainment/12. Average unrounded ratios and round total to one decimal. Any unknown category keeps the overall score null; no company means both readiness scores null.

The same hypothetical request's gain is a single-action projection, not a promise or a number that can be freely added to another suggestion. Include a reset control that sends empty override arrays/objects and restores baseline scores while the original JSON stays identical.

Second candidate: the full SQL-only input

Create samples/profile_sql.json. This complete second candidate supports the 33.3% test without another teammate's output. The name is irrelevant to the score; the reviewed SQL claim and its linked source are what matter.

```
{
  "schema_version": "TEAM-V1",
  "id": "candidate-2",
  "revision": 1,
  "name": "Dev Example (fictional)",
  "email": "dev@example.test",
  "education": ["Fictional College - computing diploma"],
  "skills": [
    {"skill_id": "cv-sql", "skill_name": "SQL", "category_code": "SQL", "evidence":
  "Skills: SQL", "confidence": "medium", "source_type": "document", "source_id":
  "resume-2", "source_page": null, "claim_status": "claimed"}
  ],
  "hackathons": [],
  "internships": [],
  "certifications": [],
  "preferred_roles": ["Junior Developer"],
  "projects": ["Built SQL reports."],
  "experience": [],
  "achievements": [],
  "cv_file": "resume-2",
  "ratings": {"COD": null, "DSA": null, "OOD": null, "APTI": null, "COMM": null, "AI":
   null, "CLOUD": null, "SQL": null, "SWE": null, "SYSD": null, "NETW": null, "OS":
  null},
  "documents": [
    {"source_id": "resume-2", "source_type": "resume", "source_file": "cv_b.docx",
  "source_text": "Name: Dev Example (fictional)\nEmail: dev@example.test\nSkills:
  SQL\nProjects: Built SQL reports.\nEducation: Fictional College - computing
  diploma"}
  ]
}
```

Second job: required and preferred reversed

Create samples/jd_b_reviewed.json. This is the complete reviewed JD B fixture. Python-only Asha now covers the preferred group, giving 33.3%, while the SQL-only candidate covers the mandatory group for 66.7%.


```
{
  "schema_version": "TEAM-V1",
  "id": "jd-2",
  "revision": 1,
  "source_type": "jd",
  "source_file": "jd_b.docx",
  "source_text": "Company: Synthetic Practice Co\nRole: Data Trainee\nRequired:
  SQL\nPreferred: Python",
  "company": "Synthetic Practice Co",
  "role": "Data Trainee",
  "name": "",
  "email": "",
  "skills": [
    {"skill_id": "b-sql", "skill_name": "SQL", "category_code": "SQL", "evidence":
  "Required: SQL", "confidence": "medium", "source_type": "document", "source_id":
  "jd-2", "source_page": null, "claim_status": "claimed"},
    {"skill_id": "b-python", "skill_name": "Python", "category_code": "COD",
  "evidence": "Preferred: Python", "confidence": "medium", "source_type": "document",
  "source_id": "jd-2", "source_page": null, "claim_status": "claimed"}
  ],
  "requirement_groups": [
    {"requirement_id": "b-required", "skill_ids": ["b-sql"], "operator": "AND",
  "requirement_type": "mandatory", "evidence": "Required: SQL"},
    {"requirement_id": "b-preferred", "skill_ids": ["b-python"], "operator": "AND",
  "requirement_type": "preferred", "evidence": "Preferred: Python"}
  ],
  "education": [],
  "projects": [],
  "experience": [],
  "internships": [],
  "hackathons": [],
  "certifications": [],
  "achievements": [],
  "review_status": "reviewed",
  "extraction_method": "manual",
  "warnings": ["Fictional expected fixture; not a live Groq result."]
}
```

Alternative job: the full AND/OR input

Create samples/jd_c_reviewed.json. Python alone scores 33.3%. In GPS, hypothetical SQL and AWS give 100%. AWS satisfies the OR group, so GCP is not also missing.


```
{
  "schema_version": "TEAM-V1",
  "id": "jd-3",
  "revision": 1,
  "source_type": "jd",
  "source_file": "jd_c.docx",
  "source_text": "Company: Synthetic Practice Co\nRole: Cloud Trainee\nRequired:
  Python and SQL\nPreferred: AWS or GCP",
  "company": "Synthetic Practice Co",
  "role": "Cloud Trainee",
  "name": "",
  "email": "",
  "skills": [
    {"skill_id": "c-python", "skill_name": "Python", "category_code": "COD",
  "evidence": "Required: Python and SQL", "confidence": "medium", "source_type":
  "document", "source_id": "jd-3", "source_page": null, "claim_status": "claimed"},
    {"skill_id": "c-sql", "skill_name": "SQL", "category_code": "SQL", "evidence":
  "Required: Python and SQL", "confidence": "medium", "source_type": "document",
  "source_id": "jd-3", "source_page": null, "claim_status": "claimed"},
    {"skill_id": "c-aws", "skill_name": "AWS", "category_code": "CLOUD", "evidence":
  "Preferred: AWS or GCP", "confidence": "medium", "source_type": "document",
  "source_id": "jd-3", "source_page": null, "claim_status": "claimed"},
    {"skill_id": "c-gcp", "skill_name": "GCP", "category_code": "CLOUD", "evidence":
  "Preferred: AWS or GCP", "confidence": "medium", "source_type": "document",
  "source_id": "jd-3", "source_page": null, "claim_status": "claimed"}
  ],
  "requirement_groups": [
    {"requirement_id": "c-required", "skill_ids": ["c-python", "c-sql"], "operator":
  "AND", "requirement_type": "mandatory", "evidence": "Required: Python and SQL"},
    {"requirement_id": "c-preferred", "skill_ids": ["c-aws", "c-gcp"], "operator":
  "OR", "requirement_type": "preferred", "evidence": "Preferred: AWS or GCP"}
  ],
  "education": [],
  "projects": [],
  "experience": [],
  "internships": [],
  "hackathons": [],
  "certifications": [],
  "achievements": [],
  "review_status": "reviewed",
  "extraction_method": "manual",
  "warnings": ["Fictional expected fixture; not a live Groq result."]
}
```

Field types and validation for your results

All listed keys in the complete examples are required. Reject unknown fields rather than silently accepting a different version. schema_version is the exact string TEAM-V1; IDs are nonblank strings, revisions integers >=1, and all numbers must be finite (true/false are not numbers). Empty arrays are allowed where shown.

Profile input, in plain language

id/revision identify the saved version. name is nonblank; email may be empty but must have an email shape if supplied. education, projects, experience, hackathons, internships, certifications, preferred_roles and achievements are lists of strings. skills is a list of the common nine-field Skill objects. ratings contains exactly the twelve category keys, each null or a number 1-10. documents holds source_id/source_type/source_file/source_text strings, and cv_file is a source_id or null. Check source links and quotes; reject hypothetical skills in persisted input.

MatchingResult and ScenarioResult

MatchingResult IDs/revisions identify the uploaded pair. jd_source_file is a string; match_score is 0-100 or null; status is complete, provisional or unscorable. matched_skills/missing_skills are lists of skill-name strings. decisions is a list of skill_name, matched (boolean), jd_evidence (string), candidate_evidence (string or null), points (number) and reason (string). group_contributions contains requirement_id, operator, requirement_type, weight, coverage and points as in the full example.

ScenarioResult hypothetical is always true. Baseline/projected match and readiness score fields and deltas are nullable numbers; readiness is null without a company or complete ratings. recommendations is a list of action, skill, projected_match_score and gain; notice is a string. skill_overrides_json is a JSON array of names; category_overrides_json is an object of valid category -> number 1-10. Empty overrides reset the scenario.

This independent file-upload API checks revision fields against the versions inside the uploaded files. It cannot detect a newer record elsewhere. Freeze the actual files for a preview and re-evaluate if they change; the Scrum Master's adapter must load authoritative current saved revisions and benchmark hashes for integrated conflict checks.

Your agreed API addresses

These are NEW TEAM-V1 teaching endpoints, not the existing completed application's endpoints. Use the exact prefix so they cannot silently collide. Base URL: http://127.0.0.1:8000. The /docs page will show the routes after you implement them.

- POST /api/team/v1/matching/evaluate - multipart: profile_file (Profile JSON), jd_file (reviewed JD Extraction JSON). Return the complete MatchingResult.
- POST /api/team/v1/matching/scenarios/preview - multipart: the SAME profile_file and jd_file, profile_revision (integer), jd_revision (integer), skill_overrides_json (JSON string array, default []), category_overrides_json (JSON object string, default {}), optional company_file (synthetic Company JSON). Return ScenarioResult; do not save.

Request and response rules

JSON body endpoints receive application/json. Upload endpoints receive multipart/form-data, a browser form that can carry file bytes. Field names are case-sensitive. Let FormData or curl set the upload boundary; do not manually force a JSON Content-Type on an upload.

A schema-valid response can still be semantically wrong. Check the source words, references and numbers. On errors return a safe message in the common shape below with an appropriate non-2xx status. 415=unsupported type; 413=too large; 422=bad fields/content; 404=not found; 409=stale revision; 502=invalid provider output; 503=provider/source/storage unavailable.

```
{
  "error": {"code": "INVALID_INPUT", "message": "Choose a readable PDF or DOCX file."}
}
```

Call your backend by hand

- 1. Start Uvicorn from the project root in one terminal. Open /docs and select the route you are testing. Click Try it out, choose the fictional files and Execute.
- 2. Inspect the response JSON. For file-pair routes, choose the complete JSON files you typed from this PDF. IDs in saved responses may differ from teaching IDs; use current returned IDs/versions.
- 3. You can also use curl.exe in a second PowerShell terminal at the root. It is written explicitly because curl can be a PowerShell alias. The long command below is ONE line; PDF wrapping is only for reading.

```
curl.exe -X POST http://127.0.0.1:8000/api/team/v1/matching/evaluate -F `
  "profile_file=@samples/profile.json" -F "jd_file=@samples/jd_reviewed.json"
```

For Modules 1/2 this command deliberately permits sending the fictional input to Groq. Do not run it against personal files without authorization. For a missing key, show a clear unavailable/manual state and leave live LLM acceptance pending.

Useful checks after you add routes

```
.\.venv\Scripts\python.exe -m pytest backend/tests -q
# From frontend/:
npm.cmd run build
```

A test is a small program with an expected result. pytest reports which assertions fail. Ask web AI to write the tests described in the milestone, inspect the expected values yourself, then run them locally. Do not replace a failing expected value with whatever the buggy code returned.

First debugging question

Does /docs return the right data? If yes, inspect frontend request/state. If no, debug the backend first. Keep the smallest failing input and the exact error. Never infer a working implementation from a nice screenshot or an HTTP 200 alone.

Resources you do and do not need

For this independent local module you need the tools and fictional JSON in this PDF. You do not need another teammate's server, the completed project, Groq or paid AI tools. All quoted people, jobs and company expectations are invented.

The organizer's target hosted storage is Supabase. Profile Builder can develop with private local JSON now; actual hosted persistence requires an authorized project, user access rules and a deliberate migration test later. Skill Matching can run without a database; it consumes exported saved inputs.

Keep original organizer-data acceptance pending until the real pack arrives. When teammates provide real exports, validate TEAM-V1, reviewed states, IDs, source links and versions; do not assume a familiar filename guarantees the right person or job.

Authorized real CV/JD testing must be separate from public examples. Personal source text travels with the teaching JSON, so treat it as sensitive. Do not push it to GitHub or paste it into web AI. Use synthetic or redacted data for debugging and screenshots.

Missing resources should not stop arithmetic, forms, persistence, API tests or UI work. Keep an honest status list: built, tested with synthetic data, mock-tested, live-verified, or pending. Never claim one status proves the others.

Build milestone 1: Load two independent input files

What and why

You should not wait for another teammate's running app to test matching.

Create or edit these files

backend/models.py, backend/inputs.py, backend/tests/test_inputs.py

What belongs in them

Define the TEAM-V1 objects and validate Profile/JD JSON uploads. Require source_type=jd and reviewed status for the JD. Check skill/category/source references and requirement group IDs. Numeric self-ratings may be null; named-JD matching does not need company benchmarks. Keep uncertain or conflicting candidate claims from earning automatic credit.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Using the complete JSON I paste, help write validation for matching inputs. No score yet. Return models.py and inputs.py separately with tests for wrong source type, broken group reference and unreviewed JD. I create files and run the commands myself.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_inputs.py -q
```

How to know it worked

The provided profile/JD validate without any live parser or database. A group referencing a nonexistent skill fails clearly.

If it goes wrong

Check exact field spelling and TEAM-V1 version. Do not silently accept a current-app 1.0 object with different fields.

Build milestone 2: Write explainable matching

What and why

The score should come from requirement coverage, not the candidate name or model opinion.

Create or edit these files

backend/matcher.py, backend/tests/test_matching.py

What belongs in them

Normalize case/whitespace and a small explicit alias map, such as JS -> JavaScript and Postgres -> PostgreSQL. Never match Java to JavaScript merely because they share a category or substring. Mandatory weight=2; preferred/unspecified=1. AND gets matched fraction; OR is covered by any member. Deduplicate identical groups, keeping strongest importance. Return decisions and points. Empty JD is unscorable/null. Uncertain claims get no credit until reviewed; fuzzy suggestions remain suggestions.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Help implement matcher.py and tests for the exact weighted formula and output below. Include aliases, AND/OR, duplicate groups, no requirements and uncertain claims. No LLM score, confidence multiplier or name-based constant. Give code I can inspect and copy, then ask me for my actual test output.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_matching.py -q
```

How to know it worked

Asha + JD A gives 66.7; SQL-only candidate + JD A gives 33.3; both skills gives 100.0. Each decision's evidence explains its points.

If it goes wrong

Recalculate weight totals on paper. Do not count every OR alternative as missing once one member satisfies that group.

Build milestone 3: Make Career GPS a safe copy

What and why

A possible future skill is not a real present claim.

Create or edit these files

backend/scenarios.py, backend/readiness.py, backend/tests/test_scenarios.py

What belongs in them

Freeze profile/JD revisions and deep-copy the inputs. Apply named skill overrides only to the copy, run the same matcher, return baseline/projected scores and reset by sending empty overrides. Rank recommendations by one-change gain, not additive promises. If a company file is supplied, use the identical readiness formula printed in this PDF for category overrides. Unknown untouched categories remain null. Never call profile save; reject mismatched revisions with 409.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Using my matcher and the scenario contract, help implement non-mutating scenarios and tests. Check original JSON equality before/after. Include SQL projection to 100%, reset to 66.7, stale revisions and optional company-readiness math. Recommendations are possibilities, not promises or measured new abilities.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_scenarios.py -q
```

How to know it worked

Hypothetical SQL changes projected match to 100; reset returns 66.7; original profile JSON is identical. A category slider alone does not create a named SQL claim.

If it goes wrong

If the original changes, find shared nested lists/dictionaries and use a genuine deep copy. Do not hide the mutation by reloading a fixture afterward.

Build milestone 4: Connect the API and React reasoning screen

What and why

Users need to see why a change helps, not only watch a number move.

Create or edit these files

backend/app.py, backend/tests/test_api.py, frontend/src/App.jsx, frontend/src/App.css

What belongs in them

Add the evaluate/preview routes and a React file-pair chooser. Show matched/missing decisions, JD and candidate quotes, contribution points, actions and a GPS baseline/projected panel. Distinguish hypothetical controls, company readiness and named-job coverage. Reset clears overrides only. Retain good inputs and show errors when a request fails.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. I will paste my matcher, scenario functions, API contract and App.jsx. Help add the exact endpoints and a simple React screen. Use FormData; show explanation chains, errors and reversible controls. Do not require another teammate server or save any hypothetical state as real ability.

Run it yourself

```
cd frontend
npm run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

How to know it worked

Load the two JSON files, see 66.7, open Python and SQL reasoning, toggle SQL for 100, then reset. Save/export the original and compare it; it must not change.

If it goes wrong

Check the payload revisions and JSON-string override fields. If only a chart moves but backend recalculation stays fixed, the simulator is not working.

Test normal cases before polishing

Create the tests yourself with web-chat help. Before asking AI for expected values, read the fictional inputs and calculate the expected meaning/math below. These are requirements for your implementation, not tests already run on code supplied with this PDF.

- Normal test 1: Asha/Python + JD A. Expected: 66.7, Python matched, SQL missing; required weight 2 and preferred weight 1 explain the points.
- Normal test 2: Different candidates/jobs. Expected: SQL-only + JD A is 33.3; Python-only + JD B is 33.3; both skills + either simple job is 100.0. Results cannot depend on names.
- Normal test 3: GPS and alternatives. Expected: Hypothetical SQL projects 100, empty overrides reset 66.7 and original stays identical. For JD C, Python alone gives 33.3; Python+SQL+AWS gives 100 (GCP is not also missing).

Check the actual browser

Run Python and React in separate terminals. Walk through one complete successful interaction and inspect the downloaded/returned JSON. Resize to about 390 px wide: labels, action buttons, quotes and comparison rows must remain readable without horizontal clipping. Use keyboard Tab and meaningful labels.

Use body text at least 16 px, a comfortable line height, clear headings, a visible primary action, loading feedback, a helpful empty state and an error that preserves useful input. Simple navy/blue on white and ordinary CSS are enough; do not spend the build phase on decorative dashboards.

Prove the answer is not hardcoded

Change a required skill or candidate rating, rename the fictional person/company without changing skills, reorder input fields and use a genuinely different document. Relevant inputs must change results; irrelevant names must not. Repeat identical scoring inputs and expect identical arithmetic. Language-model wording/confidence can vary; deterministic scoring must not.

Test failures and unfamiliar inputs

- Failure test 1: Empty JD or malformed references. Expected: Empty valid JD -> null/unscorable. Broken group IDs/invalid JSON -> 422. Never divide by zero or return a free 100%.
- Failure test 2: Conflicting / near spellings. Expected: A needs_review Python claim gets no automatic credit. Java != JavaScript. Suggested equivalence needs confirmation before it scores.
- Failure test 3: Stale or invalid scenario. Expected: Wrong profile/JD revision -> 409; category level 0/11/true -> 422. Check original deep JSON equality after success, reset and failure.

Try an unfamiliar authorized document

Use only material you own or are permitted to process. For a new JD/CV, write a small expected list before extraction: skill, category, supporting quote, section and required/preferred/alternative meaning. Count unsupported extra claims (false positives) and missed supported claims (false negatives). Inspect names/background separately. More extracted skills is not automatically better.

For Modules 3-5, make a new fictional/authorized profile or reviewed JD JSON from those facts and test the same save/compare functions. Keep ground-truth notes separate from your parser output. A successful provider call or valid JSON proves neither truth nor competency.

Ask web AI to help debug one failure

I ran this command: [paste]. I expected: [specific result]. I got: [safe output or traceback]. Here is the smallest fictional input and the full relevant file [filename]. Diagnose this one failure, explain the cause and give a small correction plus a regression test. You cannot access my computer or run the fix. Do not change the contract, weaken the assertion or ask for my .env/key.

Paste a complete error from its first useful line, not only 'it doesn't work'. If API keys or personal text appear, remove them before sharing. After a fix, rerun the failing test and the other tests; keep the observed result, including failures.

Fix common setup and API mistakes

- python/node/git not recognized: confirm installation, PATH and a newly opened terminal. Use VS Code File -> Open Folder if code . is unavailable.
- ModuleNotFoundError: use .venv's Python and run from the root containing backend/. Confirm __init__.py exists. Installing into a different Python does not fix your environment.
- Address already in use: stop your own earlier server with Ctrl+C. Do not kill an unknown process. If you choose a different port, update frontend base URL and allowed CORS origin consistently.
- Cannot reach API: open the health URL and /docs first. Frontend requests should use http://127.0.0.1:8000, not the Vite port. A server traceback is more useful than a generic browser error.
- 422 response: inspect the response body and compare exact field names/types. Files use FormData; JSON bodies use application/json. Do not send a filename string when the API expects file bytes.
- JSON parse error: use straight double quotes, no comments/trailing commas and proper escaped \n inside strings. Copy the full object, not only the visible first page.
- CORS error: compare exact protocol/host/port. localhost and 127.0.0.1 differ. Add only your development frontend origin; do not use a wildcard as a substitute for understanding the failure.
- AI gives too much code: ask for one named file and one test. If a chat limit is reached, save your work and start a new chat with this PDF, the current milestone, relevant files and safe output.

A real working module has runnable source, correct output for changed inputs, tests that can fail and a UI that calls the backend. A screenshot or prerecorded JSON alone does not meet that standard.

Put your own code on your GitHub

The PDF-only repository is the library your Scrum Master shares. Your working code belongs in a DIFFERENT repository on YOUR GitHub account. Do not push code into the guide library, overwrite someone else's remote or send a screenshot as the only deliverable.

- 1. In your project root create .gitignore with the following lines. Keep real uploads in data/private/. Check the actual staged list before committing; ignore rules cannot erase a secret already committed.

```
.venv/
.env
.env.*
!.env.example
frontend/node_modules/
frontend/dist/
__pycache__/
.pytest_cache/
data/private/
*.log
```

- 2. In GitHub's normal website choose New repository, YOUR account, a clear name and Private. Leave automatic README, license and .gitignore unchecked because you already have local work. Add your own README.md locally: prerequisites, setup/run/test commands, contract version, examples and honest limitations.

- 3. From the project root run these commands. Replace YOU/YOUR-REPO with your actual new repository URL. Git may ask for your author name/email and open browser authentication. Do not paste a key/token into an AI chat.

```
git init -b main
git add .
git status --short
git diff --cached
git commit -m "Build and test my RADIX module"
git remote add origin https://github.com/YOU/YOUR-REPO.git
git push -u origin main
```

Before the commit, inspect for keys, .env, private CVs, source text from real people, databases and large dependency folders. Commit only source, lock/requirement files, tests, fictional fixtures and safe screenshots. Never force-push. If you already initialized Git during development, skip the init and use your existing branch intentionally.

Verify a fresh checkout and send the result

First freeze the Python dependencies that actually worked. From the ROOT run the first command below. Commit backend/requirements.txt and frontend/package-lock.json; do not copy .venv or node_modules. In your README state the Python/Node versions you used.

```
.\.venv\Scripts\python.exe -m pip freeze > backend/requirements.txt
git add backend/requirements.txt frontend/package-lock.json
git commit -m "Record verified dependencies"
git push
```

Now open a new terminal in another parent directory. Clone YOUR code repository into a new folder, not on top of your working folder. This catches missing files and accidental dependence on private paths.

```
git clone https://github.com/YOU/YOUR-REPO.git fresh-check
cd fresh-check
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt
.\.venv\Scripts\python.exe -m pytest backend/tests -q
cd frontend
npm.cmd ci
npm.cmd run build
```

Start Python from the fresh root using the earlier Uvicorn command and React from fresh frontend/. Repeat one normal and one failure interaction. For Modules 1/2 add your private .env locally only if doing an authorized live test; offline/mock tests must not require a key. Do not commit that file.

Send the Scrum Master

- Your repository URL, module 5 name, and the output of git rev-parse HEAD. Invite the Scrum Master as a collaborator if the repository is private.
- Actual commands/results from the fresh checkout; three normal and three failure cases with expected/actual outcomes; input/output and error-recovery screenshots using fictional data.
- Complete TEAM-V1 output JSON and its fictional PDF/DOCX/JSON inputs in your own samples/ directory. Include changed-file notes and what is implemented, mock-tested, live-verified or pending.

You do not have to merge into the existing RADIX app or solve central conflicts. The Scrum Master evaluates your tested commit and builds the explicit bridge. Never claim your part is integrated until that separate verification succeeds.

Explain your work to judges

A short explanation to practise

I compare a candidate with one reviewed job description. The score is weighted requirement coverage, and every decision shows the job words, candidate evidence or its absence, points and a next action. Aliases are explicit and uncertain matches remain review items. Career GPS runs the same mathematics on a temporary copy. Reset restores the baseline without changing the saved candidate. A high score is evidence coverage, not a guarantee of hiring or verified competence.

Your live demo sequence

Load the profile and JD A JSON, show 66.7 and the Python/SQL evidence decisions. Preview SQL to 100, then reset and show unchanged original JSON. Try a different candidate or JD. If showing optional company readiness, use the labelled all-8 benchmark and explain that category sliders and named-skill toggles change separate calculations.

Five likely questions

- Why does Python score 66.7 here? It covers a mandatory group of weight 2 out of total weight 3; the preferred SQL group is not covered.
- Does 100% guarantee a job? No. It covers the extracted/reviewed groups; hiring and actual ability involve more evidence.
- How do fuzzy matches work? Known aliases match; uncertain near spellings are suggestions for review, not automatic credit.
- Does GPS change my real profile? No. It deep-copies inputs, recalculates and proves original JSON equality afterward.
- Is company readiness the same metric? No. It compares twelve self-ratings with a chosen benchmark; named-JD matching compares claims with requirements.

What makes it more than an AI wrapper?

The model, where used, is only one step. Validation, review, evidence traceability, persistence, transparent arithmetic, meaningful failures and non-mutating scenarios are ordinary software responsibilities that you can demonstrate and test. Show those behaviors, not just the model's paragraph.

Integration: an explicit adapter is required

This page is also for your Scrum Master. TEAM-V1 is a deliberate smaller teaching format. Your independent endpoints live under /api/team/v1/. The completed app uses /api/jd, /api/resumes, /api/profiles, /api/talent-check and /api/matching with its richer schema_version=1.0 models. Sending TEAM-V1 directly to those routes will fail validation; do not silently rename only the URL.

The bridge is NOT implemented by this PDF task. Students return working independent code and full fictional exports. The Scrum Master imports/adapts the data, registers authorized originals in the existing owner's private document store, maps IDs, then reruns existing canonical comparisons. Do not paste incoming JSON straight into the production store or mark it verified.

- Common Skill mapping: retain name/category/evidence/confidence. Map TEAM-V1 source_id to a newly registered owned source_document_id. Preserve source_type/claim_status. Add current skill_family, evidence_type, verification_status=unverified and other safe defaults. Derive/validate spans against actual source text; do not invent offsets or ownership.
- Input mapping: adapt/register the Profile and reviewed Extraction first, including the ratings-object-to-array conversion and original source links. Rerun the canonical matcher to obtain its detailed matched/missing objects, contributions, rules_version and receipt; do not pretend name arrays are that complete result.
- Scenario mapping: TEAM-V1 exposes nullable readiness score numbers, while the canonical ScenarioResult nests full readiness/matching result objects. Recompute with the canonical scenario engine and frozen saved IDs/revisions/benchmark hash. Never fill missing nested results or mutate a saved profile to imitate the preview.

Integration acceptance: same intended input facts, preserved source quotes and unknown values, consistent expected arithmetic, all five canonical-module tests and one full candidate/JD/company journey. Actual real Groq/Supabase/organizer-data tests remain separately pending until resources exist. No deployment or existing project modification is part of distributing these manuals.

Finish honestly and keep these references

Final completion checklist

- [ ] My source runs from a fresh checkout with the documented commands and dependencies.
- [ ] All organizer inputs/outputs for my role work; all project extensions marked required are demonstrated.
- [ ] Three normal and three failure cases have actual results, not only expected results.
- [ ] I exported full TEAM-V1 JSON and included fictional source fixtures and screenshots in my own code repository.
- [ ] I state what is implemented, synthetic-tested, mock-tested, live-verified and pending. I do not claim that a key or HTTP 200 proves semantic quality.
- [ ] No private key, real CV/source text, private database or organizer export is committed. The Scrum Master can access my private repository and tested commit.

Known limits

The official pack and genuine company standards remain unavailable. OCR, perfect layout support, cloud RLS/security and a live LLM evaluation are not magically supplied by this PDF. The larger completed app is preserved separately; this tutorial uses a different contract and needs its documented adapter. Do not claim integration or official-data acceptance until the Scrum Master verifies it.

Official references checked 26 October 2026

Python virtual environments

FastAPI first steps and CORS

Vite React setup

GitHub: create a repository

Organizer reference: RADIX Talent Match Participant Brief, role pages 7-8, shared contract pages 9-10 and definition of done page 13, plus the user's later read-only table announcement. Requirements are summarized here; you do not need those separate files to follow the tutorial.